CRYPTALYST

a domain model

from the first socket message to the byte on disk

Seven bounded contexts, one contract between them, and a record that can be read without the platform running.

Seven contexts · eight figures · four levels, down to the filesystem

Contents

01 How to read this

02 Level 0 — the context map

03 Level 1 — the envelope, the only thing that travels

04 Level 2 — market data, the only process that talks to the exchange

05 Level 2 — the book, the aggregate that decides

06 Level 2 — the gate, what must pass before money moves

07 Level 2 — the recorder, the only writer

08 Level 3 — the lake, what is actually on disk

09 Level 4 — beneath the lake

10 Level 2 — control, what may change behaviour

11 Crossing concerns — time, identity and generations

12 What the model deliberately refuses to do

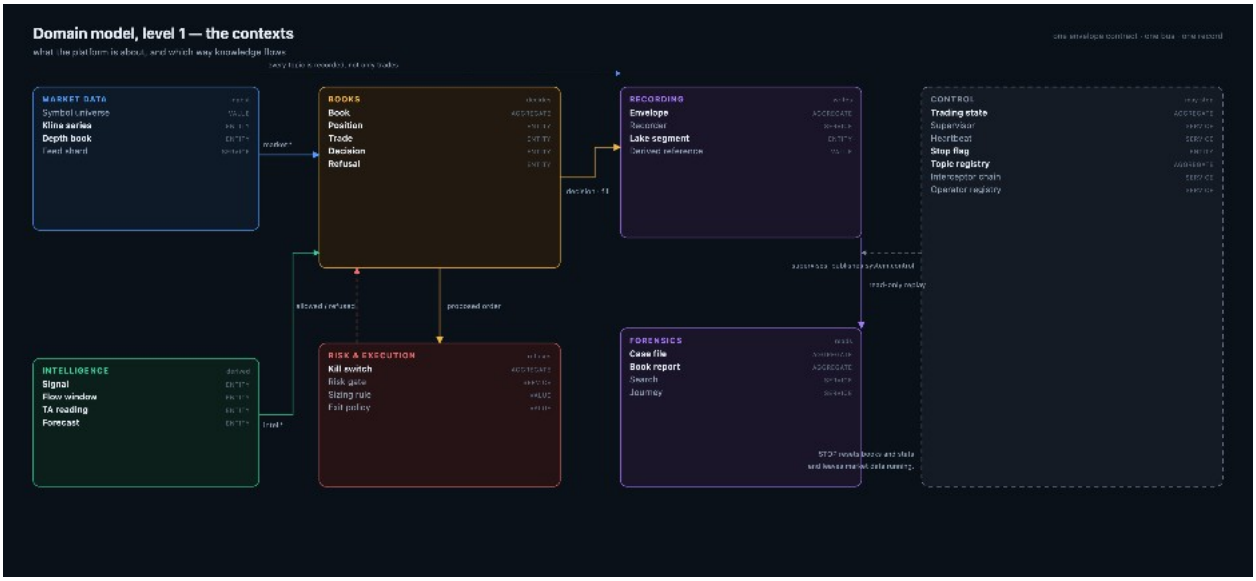
01 How to read this

A domain model is an argument about what belongs together, not an inventory of files. Each level here narrows the lens: the contexts, then one context at a time, then the record, and finally the bytes underneath the record.

Two rules run through every level. First, exactly one kind of thing travels between contexts: an envelope. Second, only the control plane may change what the platform does; data may inform a decision but never stop the machinery.

LEVEL	QUESTION IT ANSWERS	WHERE
0	what is this platform about?	the context map
1	what are the parts, and what do they own?	the seven contexts
2	what is inside one part?	a figure per context
3	what is the record, and what is beneath it?	the lake and the filesystem

02 Level 0 — the context map



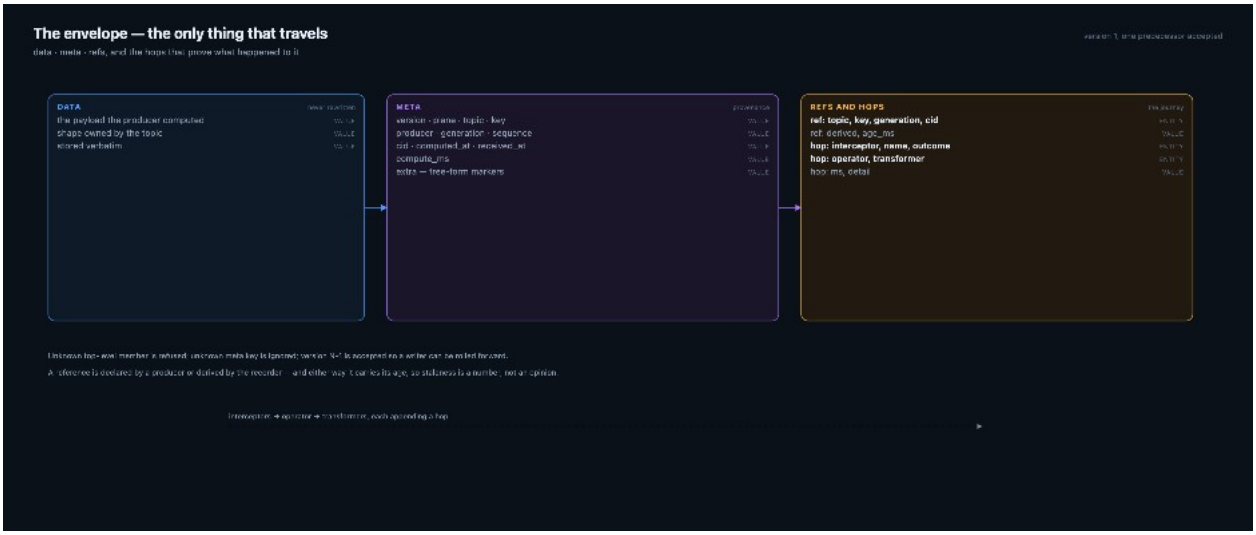
The platform in one picture: contexts, the aggregates they own, and which way knowledge flows.

Seven contexts, arranged by the direction knowledge travels. Market data observes. Intelligence interprets. Books decide. Risk refuses. Recording writes. Forensics reads. Control may stop.

The arrows are not calls. Each one is a stream of envelopes on one bus, so a context can be stopped, restarted or replaced without the others knowing.

CONTEXT	OWNS	MAY
Market data	the exchange connection, the price and depth state	publish market.*
Intelligence	signals, flow, TA, forecasts	publish intel.*
Books	equity, positions, trades, decisions, refusals	publish model.*
Risk & execution	the gate, the sizing rule, the exit policy	allow or refuse
Recording	the envelope, the journal, derived references	append to the lake
Forensics	case files, reports, search, journey	read only
Control	trading state, the stop flag, the registries	supervise, stop

03 Level 1 — the envelope, the only thing that travels



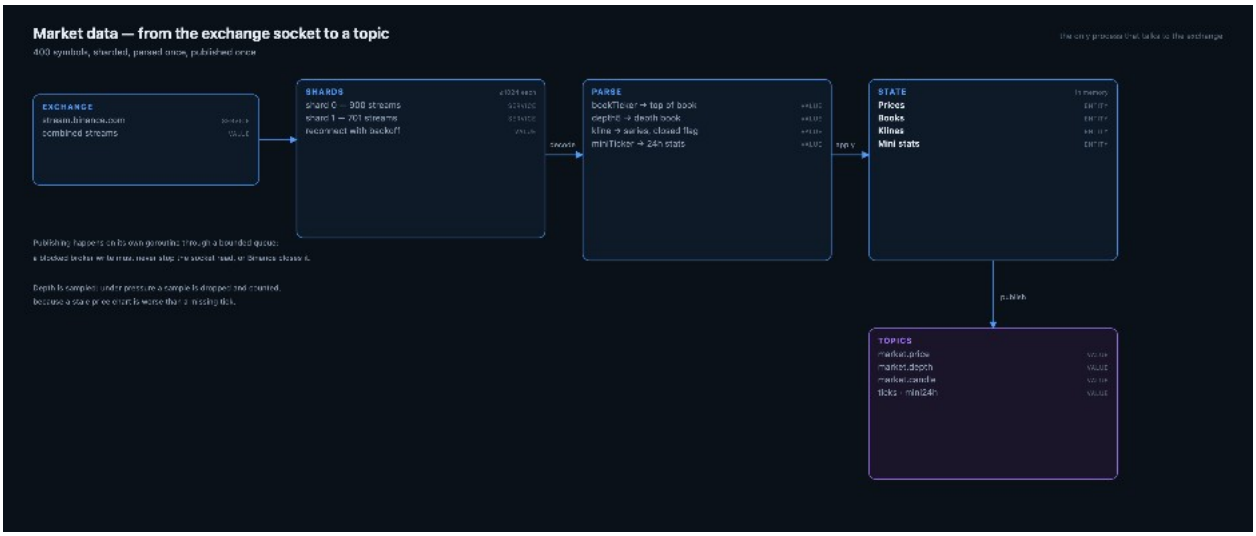
Three members, and the hops that prove what happened to the envelope on the way.

An envelope is data, meta and refs. The data is the payload a producer computed and is never rewritten. The meta is provenance: version, plane, topic, key, producer, generation, sequence, correlation id, computed_at, received_at, compute time, and a free-form extra map. The refs are the inputs, declared or derived, each carrying its age.

The contract is deliberately strict in one direction and lenient in the other. An unknown top-level member is refused, because a reader must never silently ignore something it does not understand. An unknown meta key is ignored, because provenance may grow without breaking every reader. One previous version is accepted, so a writer can be rolled forward before every reader is.

- interceptors run first and append a hop each — authority, keying, freshness, correlation, redaction
- the operator produces the payload and appends a hop
- transformers may change the shape, never the truth, and append a hop
- a refusal is a hop with outcome refuse and the rule in its detail

04 Level 2 — market data, the only process that talks to the exchange



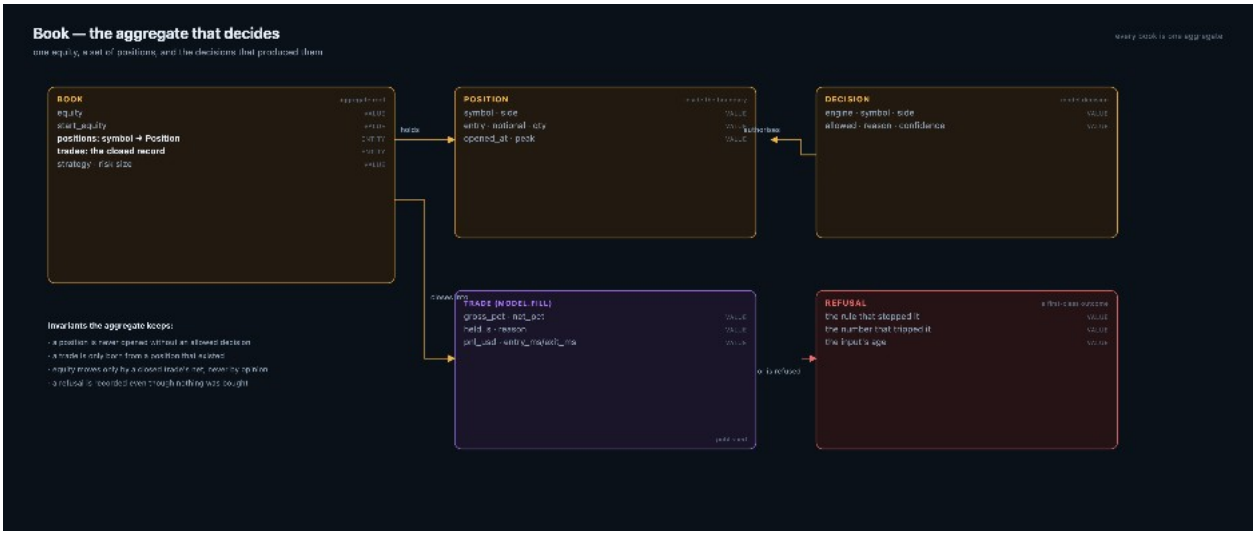
Shards, one parse, in-memory state, then topics. Publishing never blocks the socket.

The feed streams the top symbols by 24-hour quote volume, four streams each, sharded so no websocket carries more than about a thousand streams. Each shard reconnects with backoff. The parse is single-pass: bookTicker becomes top of book, depth becomes the depth book, klines become series with a closed flag, miniTicker becomes 24-hour stats.

The one design decision worth naming: publishing happens on a goroutine through a bounded queue. Publishing inline on the read path means a slow broker write stops the reader, the exchange's ping times out, and the connection dies. Under pressure the feed drops depth samples and counts them, because a missed sample is better than a dead stream.

STATE	FED BY	SERVED BY
Prices	bookTicker	GET /streamprices
Books	depth5	GET /depth?symbol=
Klines	kline_1m / kline_5m	GET /klines?symbol=&interval=
Mini stats	miniTicker	GET /movers

05 Level 2 — the book, the aggregate that decides



A book is one aggregate: equity, positions, and the decisions that produced them.

A book is the unit of consistency. Its equity moves only when a position closes, by that trade's net, and never by an opinion about an open position. Positions live inside the book; a decision that would open one either becomes an allowed decision or a refusal, and both are recorded.

The refusals are the part most systems omit. A refusal carries the rule that stopped it and the number that tripped it, so the question 'why did it not trade' is answered by the same object as 'why did it trade'.

- a position is never opened without an allowed decision
- a trade is only born from a position that existed
- equity moves only by a closed trade's net
- a refusal is recorded even though nothing was bought

06 Level 2 — the gate, what must pass before money moves



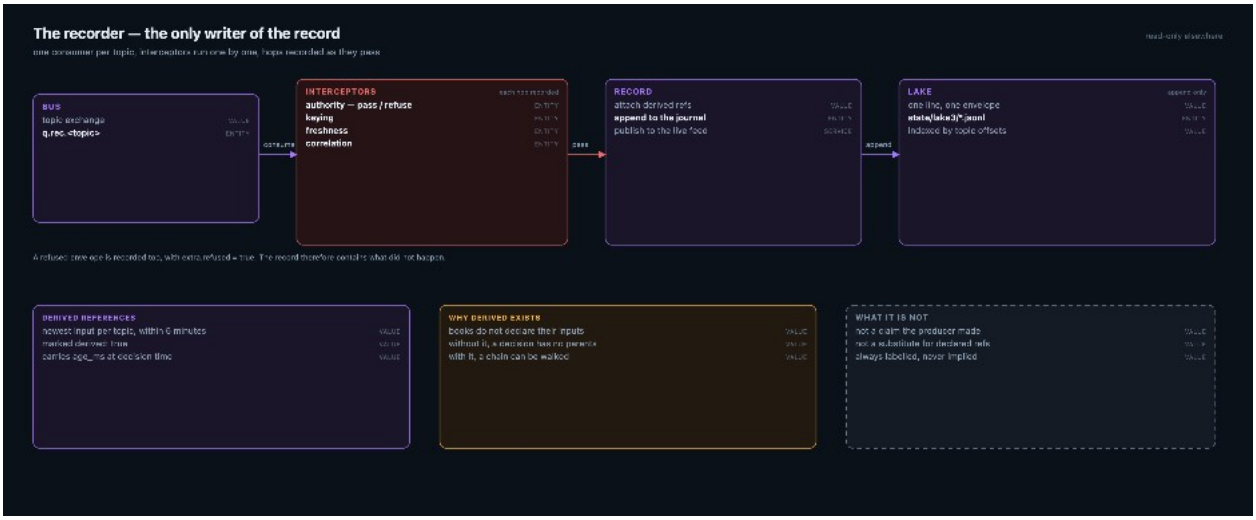
Interceptors, then the arithmetic. Every stage appends a hop, so a refusal explains itself.

The gate is ordered deliberately: identity and freshness before arithmetic. There is no point in sizing a trade on a signal that is four hours old, and no point in asking whether a stale signal is affordable.

The exit policy is part of the model, not an implementation detail. The platform once refused every losing exit, which made the fills table contain only winners while equity fell on the losers still open. The policy now holds a loss only while it is shallow and young, and releases it at a stop or a time cap. A loss is booked like any other outcome.

CHECK	REFUSES WHEN
size	notional is zero
forecast age	older than the limit
signal age	older than the limit
kill switch	the stop flag exists
gross multiple	exposure would exceed the cap
symbol share	one position would exceed the book

07 Level 2 — the recorder, the only writer



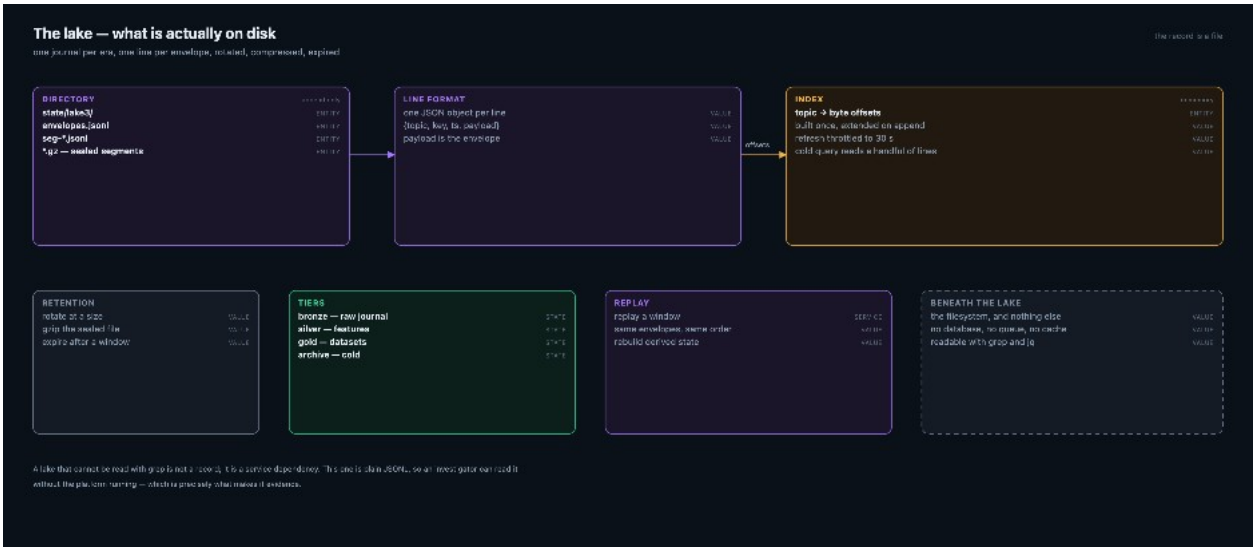
One consumer per topic, hops recorded as they pass, derived references attached at the door.

The recorder owns one queue per topic, so it sees every message exactly once. It runs the interceptors one at a time and records each as a hop on the envelope itself — which is why the pipeline an envelope went through is data, not a log line.

Books do not declare their inputs. Rather than leave a decision parentless, the recorder attaches derived references: the newest input per topic within five minutes, marked derived, carrying the age it had at decision time. It is the weakest form of lineage and it is always labelled as such.

- a refused envelope is recorded too, with extra.refused = true
- the record therefore contains what did not happen
- derived references are labelled, never implied
- the recorder declares its own topology; it does not rely on another process to do it

08 Level 3 — the lake, what is actually on disk



A journal, a line format, an index, and the retention that keeps it from growing forever.

The lake is a directory of JSONL journals. One line is one envelope. The line carries the topic, the key, the timestamp and the envelope itself. There is no database, no queue and no cache between the record and the filesystem, which is what makes it readable by grep and jq a year later.

Reader queries do not scan the journal. An index of topic to byte offsets is built once at startup and extended as the recorder appends; refreshing it is throttled, because a cold query that walks a multi-gigabyte file is indistinguishable from a hang.

LAYER	CONTENTS	WRITTEN BY
bronze	the raw journal, every envelope	the recorder
silver	features extracted from fills	the historian
gold	datasets for training and study	the trainer
archive	sealed, compressed, cold	retention

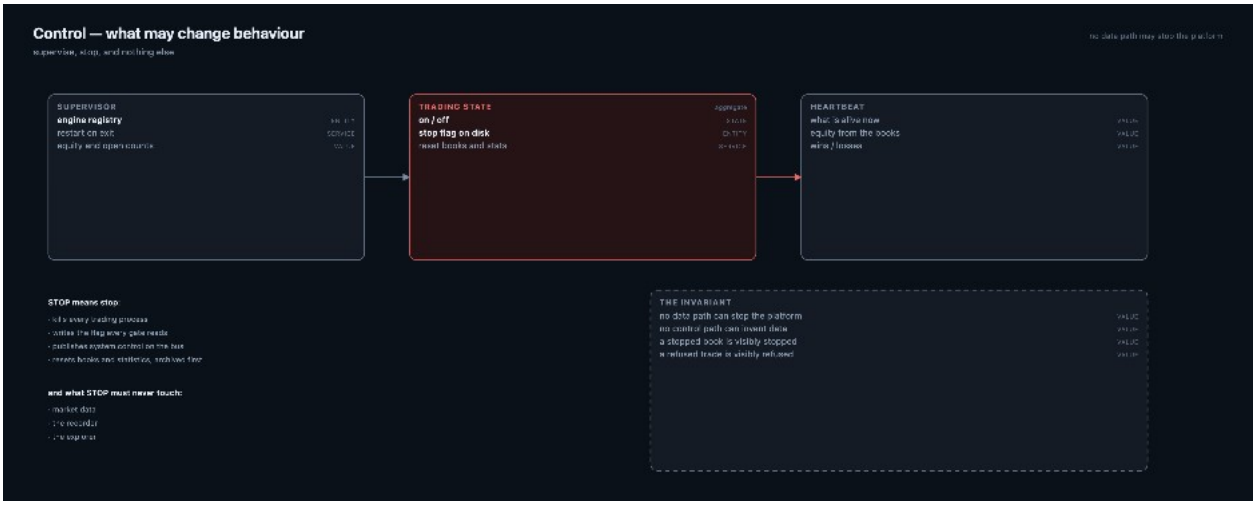
09 Level 4 — beneath the lake

Underneath the lake there is a filesystem and nothing else. The properties that follow from that are worth stating plainly, because they are the reason a case file can be called evidence.

PROPERTY	CONSEQUENCE
one line, one envelope, append only	a line never changes after it is written
plain UTF-8 JSON	grep, jq, awk and a shell are enough to read it
no service dependency to read	an investigator does not need the platform running
rotation by size, then gzip	the working journal stays small and fast
expiry by window	the record has a stated lifetime, not an accidental one
offsets in memory, rebuilt on boot	the index is disposable, the journal is not

The separation matters architecturally: the journal is the durable fact, the index is a convenience that can be thrown away and rebuilt. A corruption in the index costs a restart; a corruption in the journal costs history. Only the second one is serious, and the design keeps them apart.

10 Level 2 — control, what may change behaviour



The supervisor, the stop flag and the heartbeat, and what STOP must never touch.

STOP is the most important verb in the platform, so it is defined precisely. It kills every trading process, writes a flag that every gate reads, publishes a control envelope on the bus, and resets books and statistics after archiving them. It must never stop market data, the recorder or the explorer: an operator needs the record most at the moment they stop trading.

The inverse is equally deliberate: no data path can stop the platform. A feed outage, a dead book or a missing consumer degrades what you see; only the control plane changes what the platform will do next.

- STOP kills trading, writes the flag, publishes system.control, resets the books
- STOP leaves market data, the recorder and the explorer running
- starting the platform does not start trading unless it is asked to
- a stopped book is visibly stopped; it does not linger as a row on a dashboard

11 Crossing concerns — time, identity and generations

Three things are shared by every context and are therefore modelled once, in the envelope.

CONCERN	FIELD	RULE
when it was computed	computed_at	staleness is measured from here, never from arrival
who is asking	cid	one identity follows work across contexts
what changed	generation	a gap is a missing message, not an accident
what it read	refs	declared or derived, always with age
what happened to it	hops	every stage, with outcome and duration

A correlation id is minted locally at the boundary, so work can start without a central registry. Mapping it centrally and resolving it are separate, gated operations — a reader that cannot resolve still knows which id it was handed.

12 What the model deliberately refuses to do

A model is also a list of things it will not pretend to know. These are the boundaries, stated so they can be argued with.

- it does not reconstruct what a producer read if the producer did not say so — it derives and labels
- it does not treat a dashboard as the record; the journal is the record
- it does not let a viewer write, which is why the explorer has no write path
- it does not hide a loss: a losing exit is booked, not deferred forever
- it does not call a stopped book running, or an empty book profitable